

EECE5640 Project Proposal
Vaidehi Gohil, Anthony Yalong
March 13, 2026

For this project, we will evaluate two DSP workloads: Fast Fourier Transform (FFT) and Finite Impulse Response (FIR). Both are computationally intensive signal processing algorithms with well-understood parallel structure, making them well-suited for comparison across parallelization strategies. FFT transforms a signal from the time domain to the frequency domain; the recursive divide-and-conquer structure of the Cooley–Tukey algorithm presents natural opportunities for parallelism, particularly across independent sub-transform computations. FIR filtering applies a linear time-invariant filter to a signal via discrete convolution, where each output sample is computed as a dot product of the filter coefficients with a window of input samples, allowing independent computation of output samples. Studying both workloads allows us to compare the parallel scaling behavior of a divide-and-conquer algorithm (FFT) against one with highly independent computations (FIR) within a coherent DSP application domain.

We will use three input datasets of varying sizes to evaluate strong scaling behavior: a small synthetic audio signal, a medium synthetic audio signal, and a large real-world audio recording sourced from the LibriSpeech ASR Corpus and resampled to an appropriate length. The small and medium signals will be generated in-house. For FIR filtering, a low-pass filter will be applied to each of the three signals, with filter coefficients generated using the Hamming window method.

All experiments will be run on the Explorer Cluster at Northeastern University using the course reservation. Jobs will be submitted via SBATCH scripts, with each experiment running on a single multi-core node scaling from 1 to 32 threads. Each workload will be executed using two parallelization middlewares: Pthreads for manual thread management with fine-grained control, and OpenMP for directive-based shared-memory parallelism. This allows us to compare parallel scaling behavior as well as the programming effort and runtime overhead associated with each middleware.

For each workload and middleware combination, we will measure wall-clock execution time at 1, 2, 4, 8, 16, and 32 threads across all three input dataset sizes. Each configuration will be run 5 times, and results will be reported as mean and standard deviation. Speedup and parallel efficiency will be computed relative to a serial baseline. This yields a total of $2 \text{ workloads} \times 2 \text{ middlewares} \times 3 \text{ input sizes} \times 6 \text{ thread counts} = 72$ experimental configurations per timing run.

Wall-clock time will be measured consistently across all implementations. Intel VTune Profiler, available as a loadable module on Explorer, will be used to collect CPU utilization, thread-level parallelism, and hotspot analysis. The primary metrics collected will be execution time, speedup $S(p) = T(1) / T(p)$, parallel efficiency $E(p) = S(p) / p$, and serial fraction estimated via Amdahl's Law.

The following lists the grades that we expect to receive:

- **A** = Both workloads were implemented in both middlewares. All three input datasets were used. Timing, speedup, and efficiency were reported and thoroughly analyzed for all configurations. VTune profiling included. Write up suitable for sharing with a future employer.
- **A-** = Both workloads implemented in both middlewares. All three input datasets were used. Results reported, but analysis is less thorough.
- **B+** = Both workloads implemented in one middleware and partially implemented in the second. Two input datasets were used. Results reported and analyzed.
- **B** = One workload implemented in two middlewares. Two input datasets were used. Results reported and analyzed.
- **C** = One workload implemented in one middleware. Results reported with minimal analysis.
- **F** = Else.

Extra credit opportunities listed:

1. **MPI implementation:** Implement both workloads using MPI and compare message-passing performance against the shared-memory implementations.
2. **Additional profiling:** Use of additional Explorer profiling tools beyond VTune (e.g., perf, TAU).
3. **Weak scaling analysis:** Evaluate weak scaling by increasing problem size proportionally with thread count.